

Oefeningen

Tekst wijzigen

Voeg onderstaande HTML-code toe aan de HTML-pagina:

```
<h1>PXL</h1>
<h2>Welkom op de website van PXL</h2>
<h2>Leer programmeren bij PXL</h2>

<button>Wijzigen</button>
```

html

Requirements:

- Voeg niks toe en wijzig niks aan de HTML-code.
- Schrijf JavaScript-code die de tekst van het `<h1>` -element wijzigt naar `PXL Hogeschool` en de tekst van het tweede `<h2>` -element wijzigt naar `Word een programmeur bij PXL`.
- Zorg ervoor dat de wijzigingen pas gebeuren wanneer op de knop wordt geklikt.

▼ Hint

Om één element (de eerste match) te selecteren die overeenkomt met een bepaalde selector, kan `document.querySelector` gebruikt worden. Om alle elementen te selecteren die overeenkomen met een bepaalde selector, kan `document.querySelectorAll` gebruikt worden.

▼ Hint

Je kan een `event listener` toevoegen aan de knop om te luisteren naar het `click`-event.

```
const button = document.querySelector("button");

button.addEventListener("click", () => {
  // code om de tekst te wijzigen
});
```

js

▼ Oplossing

```
<h1>PXL</h1>
<h2>Welkom op de website van PXL</h2>
<h2>Leer programmeren bij PXL</h2>

<button>Wijzigen</button>
```

html

```
const button = document.querySelector("button");

button.addEventListener("click", () => {
  const h1Element = document.querySelector("h1");
  h1Element.textContent = "PXL Hogeschool";

  const h2Elementen = document.querySelectorAll("h2");
  const tweedeH2Element = h2Elementen[1];
  tweedeH2Element.textContent = "Word een programmeur bij PXL";
});
```

js

PXL

Welkom op de website van PXL

Leer programmeren bij PXL

Wijzigen

DOM-manipulatie + iteratie

Voeg onderstaande HTML-code toe aan de HTML-pagina:

```
<ol>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
  <li>Python</li>
  <li>Java</li>
</ol>
```

html

Voeg onderstaande CSS-code toe aan de CSS-pagina:

```
.primary {
  color: blue;
}

.secondary {
  color: green;
}
```

CSS

Schrijf JavaScript-code die alle oneven genummerde lijstitems (1, 3, 5, ...) de CSS-klasse `primary` geeft en alle even genummerde lijstitems (2, 4, 6, ...) de CSS-klasse `secondary` geeft.

▼ Hint

Je kan `document.querySelectorAll` gebruiken om alle lijstitems te selecteren. Vervolgens kan je een `for`-loop of `forEach`-methode gebruiken om door de lijstitems te itereren en de juiste CSS-klassen toe te voegen op basis van hun index.

▼ Oplossing

```
<ol>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
  <li>Python</li>
  <li>Java</li>
</ol>
```

html

```
.primary {
  color: blue;
}

.secondary {
  color: green;
}
```

CSS

```
const listItems = document.querySelectorAll('li');

listItems.forEach((item, i) => {
  if (i % 2 === 0) {
    item.classList.add('primary');
  } else {
    item.classList.add('secondary');
  }
});
```

js

```
}  
})
```

1. HTML
2. CSS
3. JavaScript
4. Python
5. Java

Extra uitdaging

De lijst moet de kleuren enkel toepassen wanneer de gebruiker over de lijst hoovert. Wanneer de gebruiker niet meer over de lijst hoovert, moeten de kleuren verdwijnen (de CSS classes moeten verwijderd worden).

▼ Oplossing

```
<ol>  
  <li>HTML</li>  
  <li>CSS</li>  
  <li>JavaScript</li>  
  <li>Python</li>  
  <li>Java</li>  
</ol>
```

html

```
.primary {  
  color: blue;  
}  
  
.secondary {  
  color: green;  
}
```

CSS

```
const list = document.querySelector("ol");  
const listItems = document.querySelectorAll("li");
```

js

```
list.addEventListener("mouseover", () => {
  listItems.forEach((item, i) => {
    if (i % 2 === 0) {
      item.classList.add("primary");
    } else {
      item.classList.add("secondary");
    }
  });
});

list.addEventListener("mouseleave", () => {
  listItems.forEach((item) => {
    item.classList.remove("primary", "secondary");
  });
});
```

1. HTML
2. CSS
3. JavaScript
4. Python
5. Java

Element tussenvoegen

Voeg onderstaande HTML-code toe aan de HTML-pagina:

```
<ul id="technologies-list">
  <li>HTML</li>
  <li>JavaScript</li>
</ul>

<button id="add-button">CSS Toevoegen</button>
```

html

Schrijf JavaScript-code die een nieuw `li`-item met de tekst `CSS` toevoegt aan de ongeordende lijst wanneer op de knop wordt geklikt. Dit

item moet tussen `HTML` en `JavaScript` worden toegevoegd.

Gebruik Google of een LLM om een manier te vinden om een nieuw element op een specifieke positie in de lijst toe te voegen. Indien er gekozen wordt voor een LLM, zoek dan vervolgens ook naar de documentatie via Google om de suggestie te valideren.

▼ Hint

Het laatste item kan opgevraagd worden met `list.lastElementChild`, er van uitgaande dat de `ul` is opgeslagen in de variabele `list`.

```
const list = document.querySelector("#technologies-list");
const lastItem = list.lastElementChild;
```

js

▼ Oplossing

```
<ul id="technologies-list">
  <li>HTML</li>
  <li>JavaScript</li>
</ul>

<button id="add-button">CSS Tussenvoegen</button>
```

html

```
const button = document.querySelector("#add-button");

button.addEventListener("click", () => {
  const list = document.querySelector("#technologies-list");

  const newItem = document.createElement("li");
  newItem.textContent = "CSS";

  const lastItem = list.lastElementChild;

  list.insertBefore(newItem, lastItem);
});
```

js

- HTML
- JavaScript

CSS Tussenvoegen

Template gebruiken om een item toe te voegen

Voeg onderstaande HTML-code toe aan de HTML-pagina:

```
html
<h1>Mijn favoriete Pokémon</h1>
<p>Inderdaad, Pokémon is zowel het enkelvoud als het
meervoud.</p>

<ul id="pokemon-list"></ul>

<template id="pokemon-item">
  <li>
    <h2 class="pokemon-name"></h2>
    <img class="pokemon-image" src="" alt="Pokémon
afbeelding" />
  </li>
</template>
```

Voeg onderstaande array toe aan het JavaScript-bestand:

```
js
const pokemonList = [
  {
    name: "Pikachu",
    image:
```



```

    "https://raw.githubusercontent.com/PokeAPI/sprites/master/spr
  },
  {
    name: "Charmander",
    image:

    "https://raw.githubusercontent.com/PokeAPI/sprites/master/spr
  },
  {
    name: "Bulbasaur",
    image:

    "https://raw.githubusercontent.com/PokeAPI/sprites/master/spr
  },
  {
    name: "Squirtle",
    image:

    "https://raw.githubusercontent.com/PokeAPI/sprites/master/spr
  },
];

```

Schrijf JavaScript-code die itereert over de array. Voor elk item in de array moet een nieuw `li`-item worden aangemaakt op basis van de `template` die klaargezet is in de HTML. De naam en afbeelding van de Pokémon moeten worden ingevuld in het nieuwe `li`-item. Vervolgens moet het nieuwe `li`-item worden toegevoegd aan het `ul`-element.

template

Een `template`-element is een manier om HTML-structuren te definiëren die **niet getoond** worden op de pagina. Via JavaScript kan de inhoud van een `template` gekopieerd worden. Vervolgens kan de inhoud gewijzigd worden om daarna het element met de gewijzigde inhoud toe te voegen aan de DOM.

```

const template = document.querySelector("#pokemon-item");
const clone = template.content.cloneNode(true);

```

js

```

/*
 * Het clone-element bevat nu een kopie van de inhoud van de
 template.
 * In dit geval een <li>-element met daarin een <h2> en een
 <img>.
 *
 * Het is mogelijk om children van een element op te vragen
 via querySelector.
 */
clone.querySelector(".pokemon-name").textContent = "Pikachu";

```

▼ Oplossing

html

```

<h1>Mijn favoriete Pokémon</h1>
<p>Inderdaad, Pokémon is zowel het enkelvoud als het
meervoud.</p>

<ul id="pokemon-list"></ul>

<template id="pokemon-item">
  <li>
    <h2 class="pokemon-name"></h2>
    <img class="pokemon-image" src="" alt="Pokémon
afbeelding">
  </li>
</template>

```

js

```

const pokemonList = [
  {
    name: "Pikachu",
    image:

"https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites
  },
  {
    name: "Charmander",
    image:

```

```
"https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites
  },
  {
    name: "Bulbasaur",
    image:

"https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites
  },
  {
    name: "Squirtle",
    image:

"https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites
  },
];

const pokemonListElement = document.querySelector("#pokemon-list")
const pokemonTemplate = document.querySelector("#pokemon-item");

pokemonList.forEach((pokemon) => {
  const clone = pokemonTemplate.content.cloneNode(true);

  clone.querySelector(".pokemon-name").textContent = pokemon.name
  clone.querySelector(".pokemon-image").src = pokemon.image;
  clone.querySelector(".pokemon-image").alt = `${pokemon.name} af

  pokemonListElement.appendChild(clone);
});
```

Mijn favoriete Pokémon

Inderdaad, Pokémon is zowel het enkelvoud als het meervoud.

- **Pikachu**



- **Charmander**



- **Bulbasaur**

Takenlijst

Voeg onderstaande HTML-code toe aan de HTML-pagina:

```
<header>
  <h1>Mijn Takenlijst</h1>
</header>

<main>
  <form>
    <input type="text" id="task-input"
placeholder="Nieuwe taak" />
    <button id="add-task-button">Taak Toevoegen</button>
```

html

```
</form>

<ul id="task-list"></ul>
</main>
```

Voeg onderstaande CSS-code toe aan de CSS-pagina:

```
.completed {
  text-decoration: line-through;
  color: gray;
}
ul {
  list-style: none;
  padding-left: 0;
}
li {
  display: flex;
  align-items: center;
  gap: 10px;
  margin-bottom: 5px;
}
```

CSS

form - preventDefault

Om te voorkomen dat het formulier de pagina herlaadt bij het indienen (het **default** -gedrag van een formulier), kan er een **event listener** worden toegevoegen aan het **form** -element waarbij er naar het **submit** -event wordt geluisterd.

Op het **event** -object kan de **preventDefault()** -methode worden aangeroepen om het **default** -gedrag te voorkomen.

```
const form = document.querySelector("form");
form.addEventListener("submit", (event) => {
  event.preventDefault();
});
```

js

```
// code om de taak toe te voegen  
});
```

De volgende functionaliteiten moeten geïmplementeerd worden met JavaScript:

- Wanneer de gebruiker een taak invoert in het tekstveld en op de knop **Taak Toevoegen** klikt, moet de taak worden toegevoegd als nieuw **li**-item in het **ul**-element.
- Elk **li**-item moet een checkbox bevatten. Wanneer de checkbox wordt aangevinkt, moet de bijbehorende taak als voltooid worden gemarkeerd door de tekst door te strepen (CSS-klasse **completed** toevoegen).
- Wanneer een taak is gemarkeerd als voltooid, moet het mogelijk zijn om de taak weer als onvoltooid te markeren door de checkbox uit te vinken (CSS-klasse **completed** verwijderen).
- Maak het input-veld leeg nadat een taak is toegevoegd.
- Voeg een knop **Verwijderen** toe aan elk **li**-item. Wanneer op deze knop wordt geklikt, moet de bijbehorende taak uit de lijst worden verwijderd.
- Validatie: **trim** de ingevoerde tekst en voorkom dat lege taken worden toegevoegd aan de lijst.

▼ Oplossing

```
<header>  
  <h1>Mijn Takenlijst</h1>  
</header>  
  
<main>  
  <form>  
    <input type="text" id="task-input" placeholder="Nieuwe  
taak">  
    <button id="add-task-button">Taak Toevoegen</button>  
  </form>
```

html

```
<ul id="task-list"></ul>
</main>
```

js

```
const form = document.querySelector("form");
const input = document.querySelector("#task-input");
const taskList = document.querySelector("#task-list");

function createTaskElement(taskText) {
  const li = document.createElement("li");

  const checkbox = document.createElement("input");
  checkbox.type = "checkbox";
  checkbox.addEventListener("change", () => {
    li.classList.toggle("completed");
  });

  const span = document.createElement("span");
  span.textContent = taskText;

  const deleteButton = document.createElement("button");
  deleteButton.textContent = "Verwijderen";
  deleteButton.addEventListener("click", () => {
    li.remove();
  });

  li.appendChild(checkbox);
  li.appendChild(span);
  li.appendChild(deleteButton);

  return li;
}

form.addEventListener("submit", (event) => {
  event.preventDefault();

  const taskText = input.value.trim();
  if (taskText === "") return;

  const taskElement = createTaskElement(taskText);
  taskList.appendChild(taskElement);

  input.value = "";
});
```

```
input.focus();  
});
```

Mijn Takenlijst

localStorage

In de browser is er een `localStorage` -object beschikbaar waarmee gegevens in een browser opgeslagen kunnen worden. Gegevens die in `localStorage` worden opgeslagen, zijn opnieuw beschikbaar wanneer de gebruiker de pagina herlaadt of de browser opnieuw opent.

Informatie opslaan in localStorage

Voeg onderstaande HTML-code toe aan de HTML-pagina:

```
<form>  
  <label for="name-input">Naam:</label>  
  <input  
    type="text"  
    id="name-input"  
    name="name"
```

html


```

        placeholder="Voer je naam in"
    />
    <button type="submit" id="save-button">Opslaan</button>
</form>

<p id="greeting"></p>

```

Schrijf JavaScript-code die de volgende functionaliteiten implementeert:

- Wanneer de gebruiker een naam invoert in het tekstveld en op de knop `Opslaan` klikt, moet de naam worden opgeslagen in `localStorage` met als key `username`.
- Wanneer de pagina wordt geladen, moet de naam worden opgehaald uit `localStorage`. Indien er een naam is opgeslagen, moet er een begroeting worden weergegeven in het `<p>`-element met de tekst `Hallo, [naam]!` (waarbij `[naam]` de opgeslagen naam is). Indien er geen naam is opgeslagen, moet het `<p>`-element leeg blijven.

preventDefault

Om te voorkomen dat het formulier de pagina herlaadt bij het klikken op de knop, kan `preventDefault()` worden aangeroepen op het `submit`-event van het formulier.

▼ Oplossing

```

html
<form>
  <label for="name-input">Naam:</label>
  <input type="text" id="name-input" name="name"
placeholder="Voer je naam in">
  <button type="submit" id="save-button">Opslaan</button>
</form>

<p id="greeting"></p>

```

```

js
const form = document.querySelector("form");
const nameInput = document.querySelector("#name-input");

```

```
const greetingParagraph =
document.querySelector("#greeting");

function displayGreeting() {
  const storedName = localStorage.getItem("username");
  if (storedName) {
    greetingParagraph.textContent = `Hallo, ${storedName}!`;
  } else {
    greetingParagraph.textContent = "";
  }
}

form.addEventListener("submit", (event) => {
  event.preventDefault();

  const name = nameInput.value.trim();
  if (name) {
    localStorage.setItem("username", name);
    displayGreeting();
    nameInput.value = "";
  }
});

displayGreeting();
```

Naam:

Informatie verwijderen uit localStorage

Breid de vorige oefening uit met een extra knop **Verwijderen**. Wanneer op deze knop wordt geklikt, moet de opgeslagen naam worden verwijderd uit **localStorage** en het **<p>**-element moet leeg worden gemaakt.

▼ Oplossing

```
html
<form>
  <label for="name-input">Naam:</label>
  <input type="text" id="name-input" name="name"
placeholder="Voer je naam in">
  <button type="submit" id="save-button">Opslaan</button>
  <button type="button" id="delete-button">Verwijder uit
localStorage</button>
</form>

<p id="greeting"></p>
```

```
js
const form = document.querySelector("form");
const nameInput = document.querySelector("#name-input");
const greetingParagraph =
document.querySelector("#greeting");
const deleteButton = document.querySelector("#delete-
button");

function displayGreeting() {
  const storedName = localStorage.getItem("username");
  if (storedName) {
    greetingParagraph.textContent = `Hallo, ${storedName}!`;
  } else {
    greetingParagraph.textContent = "";
  }
}

form.addEventListener("submit", (event) => {
  event.preventDefault();

  const name = nameInput.value.trim();
  if (name) {
    localStorage.setItem("username", name);
  }
});
```

```
        displayGreeting();
        nameInput.value = "";
    }
});

deleteButton.addEventListener("click", () => {
    localStorage.removeItem("username");
    displayGreeting();
});

displayGreeting();
```

Naam:

Complexe gegevens opslaan in localStorage

Gegevens in `localStorage` worden altijd opgeslagen als `string`. Om complexe gegevens zoals arrays of objecten op te slaan, moeten deze eerst worden omgezet naar een string. Bij het ophalen van de gegevens moeten ze weer omgezet worden naar een array of object.

Het omzetten van een array of object naar een string, kan met `JSON.stringify()`. Het omzetten van een string naar een array of object, kan met `JSON.parse()`.

```
const persoon = {
  voornaam: "John",
```

js

```
    achternaam: "Duck",
    leeftijd: 30,
  };

  const persoonString = JSON.stringify(persoon);
  console.log(persoonString); //
  '{"voornaam":"John","achternaam":"Duck","leeftijd":30}'
  console.log(typeof persoonString); // "string"

  const persoonObject = JSON.parse(persoonString);
  console.log(persoonObject); // { voornaam: "John",
  achternaam: "Duck", leeftijd: 30 }
  console.log(typeof persoonObject); // "object"
```

Breid de takenlijst uit de andere oefening uit met de volgende functionaliteiten:

- Sla de takenlijst op in `localStorage` telkens wanneer er een taak wordt toegevoegd, verwijderd of gemarkeerd als voltooid/onvoltooid.
- Wanneer de pagina wordt geladen, moet de takenlijst worden ingeladen vanuit `localStorage` en weergegeven worden in de lijst.

[Previous page](#)

[Theorie](#)